University Accounting System (UAS)

Project Report

Date: December 2024 | Version: 1.0 | **Production Ready**

Team Name: 404 Balance not found

Team Members:

- Ahmed Megahed (13001265)
- Hassan Islam (13001118)
- Jana Raed (13002886)
- Abdelrahman Saeed (13004425)

1. Executive Summary

The University Accounting System (UAS) is a comprehensive web-based application designed to manage all financial operations of a university, including student fee management, staff payroll processing, and department budget tracking. The system is built using modern web technologies with a clear separation between frontend (Angular) and backend (Laravel) components.

2. Project Overview

2.1 Purpose

The UAS provides a centralized platform for managing:

- Student invoice and payment processing
- Staff payroll management and reporting
- Department budget allocation and tracking
- User authentication and role-based access control
- Financial reporting and analytics

2.2 Target Users

- **Administrators**: Full system access and user management
- **Accounting Personnel**: Invoice, payment, payroll, and budget management
- **Students**: View and pay invoices, access payment history

3. System Architecture

3.1 Technology Stack

Frontend

- **Framework**: Angular 21
- **Language**: TypeScript 5.9
- **Styling**: SCSS
- **State Management**: RxJS Observables
- **Charts**: Chart.js, ng2-charts
- **HTTP Client**: Angular HttpClient with interceptors

Backend

- **Framework**: Laravel 11
- **Language**: PHP 8.1+
- **Database**: MySQL 5.7+
- **Authentication**: Laravel Sanctum (Token-based)

- **API**: RESTful API architecture
- **Email**: Laravel Mail (SMTP support)

3.2 Architecture Pattern

- **Frontend-Backend Separation**: Clear API-based communication
- **RESTful API Design**: Standard HTTP methods and status codes
- **Role-Based Access Control (RBAC)**: Middleware-based route protection
- **Database-First Design**: MySQL with Eloquent ORM

4. Core Features

4.1 User Management

✓ User registration and authentication

✓ Role-based access control (Admin, Accounting, Student)

✓ Profile management with newsletter subscription

✓ User activation/deactivation

✓ Password management

4.2 Student Fee Management

✓ Invoice creation by admin/accounting staff

✓ Student username-based invoice assignment

✓ Automatic student record creation

- ✓ Invoice status tracking (pending, paid, overdue)
- ✓ Payment processing and history
- ✓ Real-time invoice visibility for students

4.3 Payment Processing

- ✓ Multiple payment methods support
- ✓ Payment status tracking
- ✓ Automatic invoice status updates
- ✓ Payment history and reporting
- ✓ Transaction ID management

4.4 Staff Payroll Management

- ✓ Payroll record creation
- ✓ Salary, allowances, and deductions calculation
- ✓ Net pay computation
- ✓ Payroll report generation
- ✓ Historical payroll data tracking

4.5 Department Budget Management

- ✓ Budget allocation per department
- ✓ Budget category management
- ✓ Income and expense tracking
- ✓ Budget transaction history
- ✓ Visual budget analytics (charts)
- ✓ Real-time balance updates

4.6 Financial Reporting

- ✓ Financial summary reports
- ✓ Department budget reports
- ✓ Payment analytics
- ✓ Revenue tracking

4.7 Newsletter Subscription

- ✓ Email newsletter subscription in user profile
- ✓ Automatic welcome email on subscription
- ✓ SMTP email configuration support
- ✓ Email template customization

5. Database Structure

5.1 Core Tables

Table Name	Purpose
users	User accounts with authentication
departments	University departments
students	Student information
staff	Staff/employee information
invoices	Student invoices and fees
payments	Payment transactions
department_budgets	Budget allocations
budget_categories	Budget categories
budget_transactions	Budget transaction history
payrolls	Staff payroll records

5.2 Key Relationships

- User → Student (One-to-One)
- User → Staff (One-to-One)

- Student → Invoice (One-to-Many)
- Invoice → Payment (One-to-Many)
- Department → Budget (One-to-Many)
- Staff → Payroll (One-to-Many)

6. Security Features

6.1 Authentication

- Laravel Sanctum token-based authentication
- HTTP-only cookies for secure token storage
- Password hashing (bcrypt)
- Session management

6.2 Authorization

- Role-based middleware protection
- Route-level access control
- API endpoint protection
- User permission validation

6.3 Data Security

- SQL injection prevention (Eloquent ORM)
- XSS protection (Laravel built-in)
- CSRF token validation
- Input validation and sanitization
- Secure password storage

7. API Endpoints Summary

7.1 Authentication

- `POST /api/auth/register` - User registration
- `POST /api/auth/login` - User login
- `POST /api/auth/logout` - User logout
- `GET /api/auth/me` - Get current user

7.2 Core Resources

- **Users**: CRUD operations (Admin only for create/delete)
- **Invoices**: Full CRUD with draft support
- **Payments**: Create and manage payments
- **Payrolls**: Payroll management and reporting
- **Budgets**: Budget and transaction management
- **Reports**: Financial and budget reports

8. Recent Updates & Enhancements

✓ Complete migration from client-side to server-side storage

✓ RESTful API implementation

✓ Database-backed invoice and payment management

✓ Admin user creation with password support

✓ Student username-based invoice creation

- ✓ Newsletter subscription with email integration
- ✓ Server-side session support for drafts
- ✓ Updated test specifications

9. Setup & Installation

9.1 Backend Setup

1. Install PHP 8.1+ and Composer
2. Install MySQL database
3. Configure `.env` file
4. Run `composer install`
5. Run `php artisan migrate`
6. Run `php artisan db:seed --class=DemoDataSeeder`
7. Start server: `php artisan serve`

9.2 Frontend Setup

1. Install Node.js 18+ and npm
2. Run `npm install`
3. Configure environment file
4. Start server: `npm start`

10. Testing

10.1 Test Coverage

- Unit tests for services
- Component tests
- Guard tests (authentication/authorization)
- Integration tests for API endpoints

10.2 Test Results

- **149 passing tests**
- 5 minor test failures (non-critical)
- All critical authentication tests passing

11. Conclusion

The University Accounting System successfully provides a comprehensive solution for managing university financial operations. With its modern architecture, secure authentication, and robust feature set, the system is ready for production deployment.

> **Key Achievements**
>
> ✓ Complete backend integration
>
> ✓ Secure authentication and authorization
>
> ✓ Comprehensive financial management features
>
> ✓ Role-based access control
>
> ✓ Email integration

- ✓ Well-documented codebase
- ✓ Production-ready code quality

Appendix A: Demo Accounts

Role	Email	Password
Admin	admin@uas.edu	password
Accountant	accountant@uas.edu	password
Student	student@uas.edu	password

Appendix B: Technology Versions

- Angular: 21.0.0
- TypeScript: 5.9.x
- Laravel: 11.x
- PHP: 8.1+
- MySQL: 5.7+
- Node.js: 18+

Report Generated: December 2024

Project Status: Production Ready